Internet OF Things CMM018

Automatic Fish Feeder

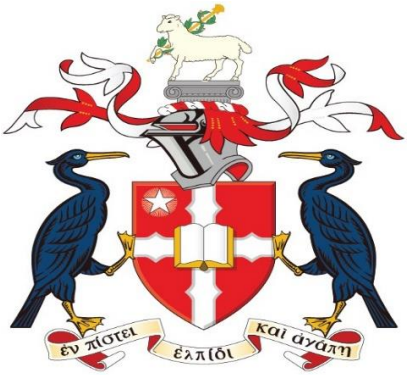
Sahar Mirzabaki

Student ID: 23011185

Module Leader: DR. Hamzah AlZu'bi

MSc Advanced Computer Science

7 JUNE 2024



LIVERPOOL HOPE UNIVERSITY

1844

Product Idea:

In this project I designed an automatic fish feeder with a new structure. This IoT project aims to provide convenience and reliability to aquarium enthusiasts, for feeding their fish on schedule and also change the schedule easily using the scheduler in a web application. Using an ESP32 microcontroller and a real-time database, users can remotely manage feeding times and check the feeder's status in real-time.

Literature Review and Market Research:

Automated fish feeders are common, but most they do not have features for remote operation or real-time monitoring. Products like the Eheim Everyday Fish Feeder offer basic automation but lack remote management and dynamic scheduling.

Advancements in IoT technology now allow for adding internet connectivity, cloud-based data storage, and mobile app integration to fish feeders. Market research shows a growing demand for smart home devices that make pet care easier, indicating a potential market for advanced fish feeders.

This device addresses the shortcomings of current products by using modern IoT features. This makes it a convenient and user-friendly solution, fitting well with the trend of smart home automation. The project's use of real-time data management and remote control sets it apart from traditional fish feeders, highlighting its potential to improve pet care through technology.

Here are some examples of different samples of fish feeders in market:

1- Eheim Every day fish feeder
2- Fish Mate F14
3- Pet Safe 5-meal automatic Pet feeder

Each of these models have different features that is mentioned bellows:

Eheim Everyday Fish Feeder Features:

Features: Basic automation, adjustable feeding times, simple to use.

Limitations: No remote control, no real-time monitoring, manual scheduling required.

Price Range: typically, around $20-$30.[4]

Fish Mate F14 Features:

Features: Programmable for multiple daily feedings, battery-operated, compact design.

Limitations: Lacks internet connectivity, no mobile app integration, limited to predefined schedules.

Price Range: typically, around $30-$40.[6]

Pet Safe 5-Meal Automatic Pet Feeder:

Features: Five-meal capacity, programmable, suitable for various pets.

Limitations: No IoT capabilities, manual programming, no real-time status updates.

Price Range: typically, around $50-$60.[5]

Hardware & Software Components:

Hardware Components

ESP32 Board: The core microcontroller for handling Wi-Fi connectivity and controlling the servo motor.

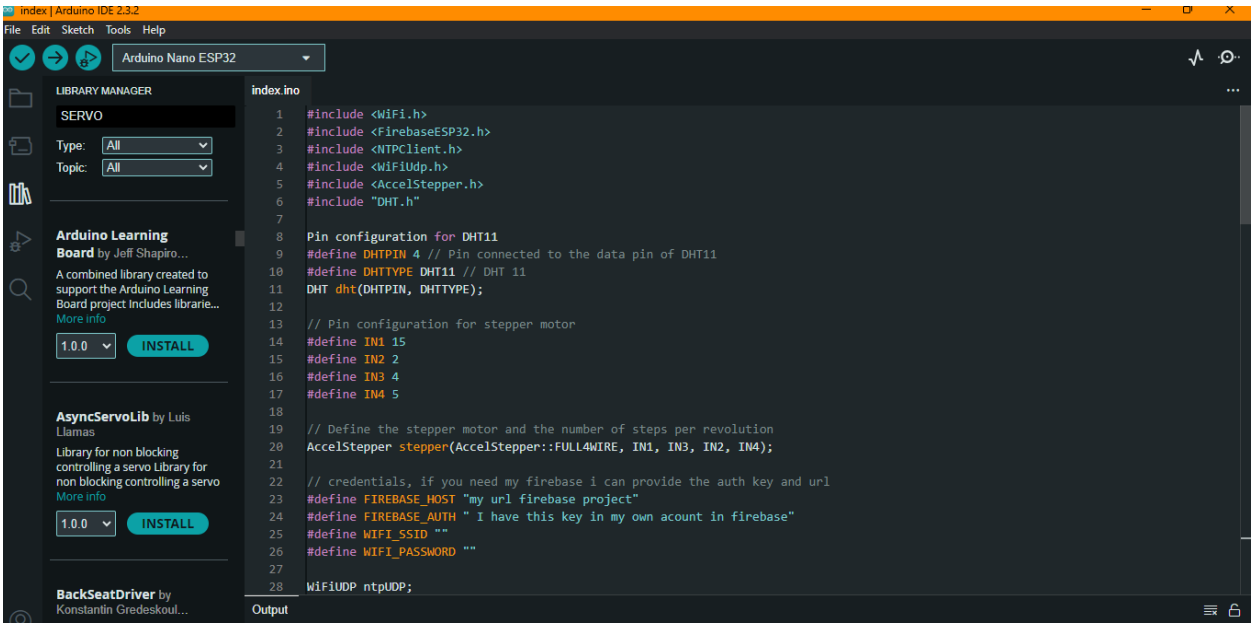
Micro Servo SG90: Used to dispense the fish food.

Instead of Servo Motor SG90 I also I tried to used Stepper motor: this motor compare to the previous one is more powerful. I wrote the code for both of them but there is some little errors.

Power Supply: 5V 2A USB adapter to power the ESP32 and servo motor.

Software Components:

Arduino IDE: I Used Arduino IDE for programming the ESP32.[1]. I installed some libraries like in this library such as: WIFI, SERVO, FIREBASE ESP32, NTP Client and FirebaseESP32



I selected the correct board from the tools which here I selected the ESP32 Arduino. Also, I selected the suitable port for my Arduino board.

Firebase services: this website provides Real-time databases for different purposes and it belongs to the Google company.[2]

NTP Client: this library is used for synchronize the time for scheduled feedings. [2]

Web Site: I developed and design a simple web site for scheduling the timer and set the feeding time. This web site has a simple and also, beautiful user interface and user interaction.

furthermore, I sued some technologies like HTML, CSS and, JavaScript Programing Languages.[7],[8],[9]

Implementation:

Step 1: create a new Sketch file and wrote the code to connecting my ESP32 Board into the firebase Database and internet thorough the WIFI protocol.

In the above code I configured the firebase database and, WIFI credentials such as:

1. FIREBASE_AUTH,
2. FIREBASE_HOST,
3. WIFI _SSID,
4. WIFI_PASSWORD,

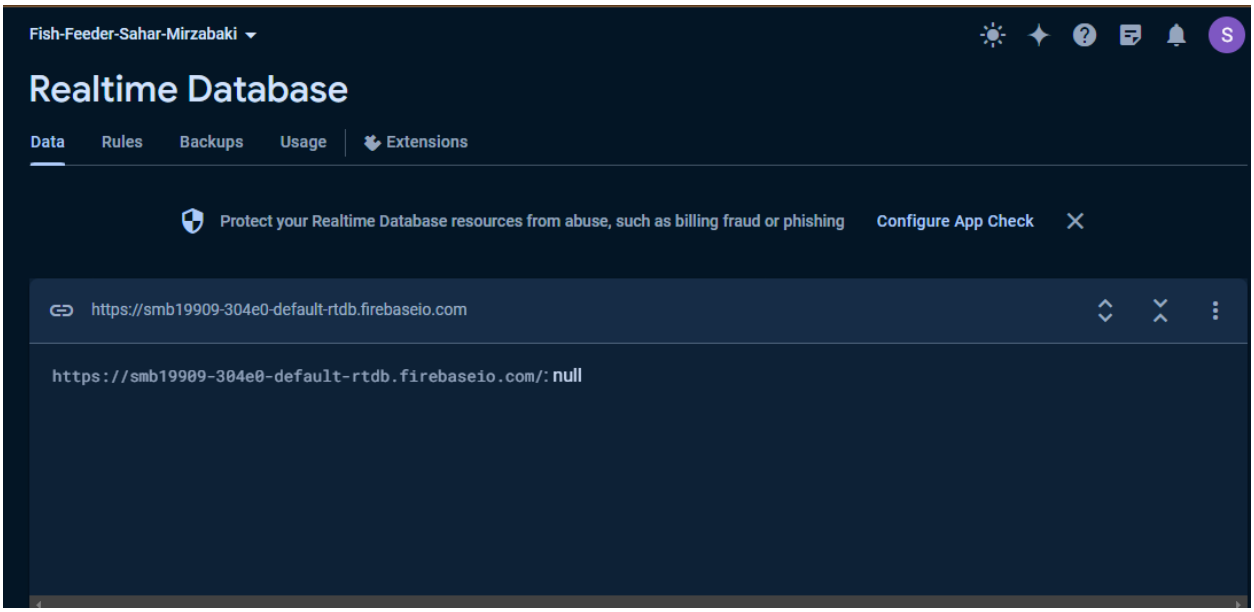
The first one and second one is related to the real time database including the server and API.

The third one and forth one is related to the WIFI protocol which is used for my IOT project.

Step 2: I used firebase Database website and created a real time database which is used as the server and, also API for my project.

For setting firebase auth, I used this part which is related to the database services:

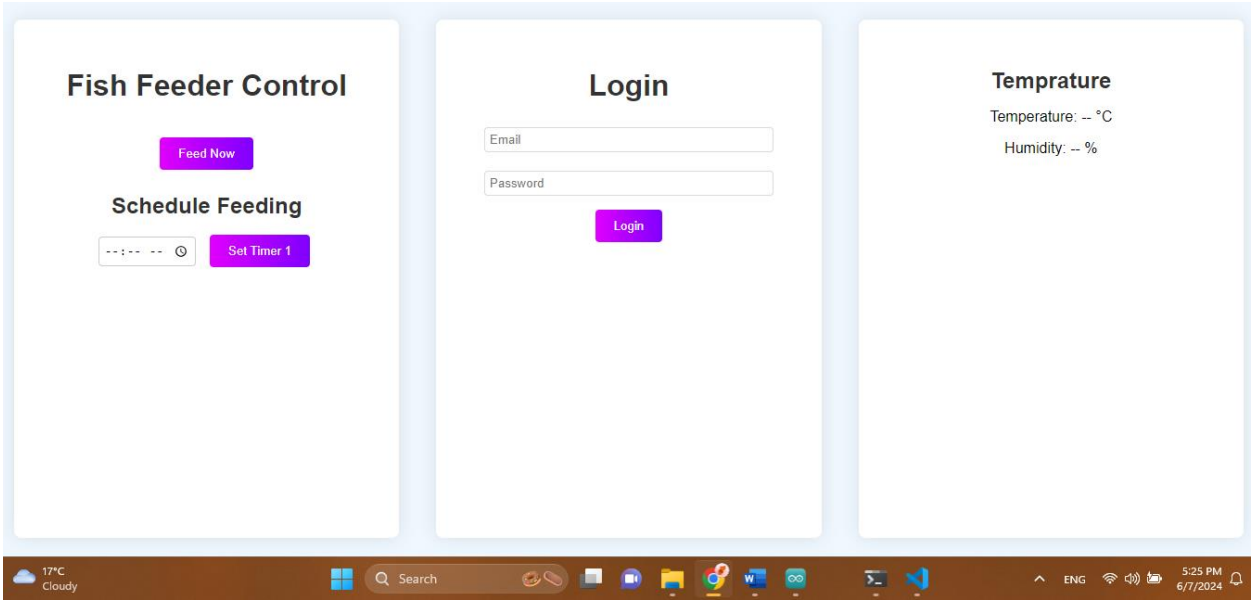
For the firebase host I used my URL in the real time database in the firebase website.



Step 3: I installed the firebase tools on my local system with cmd command prompt for more accessibility to the firebase tools.

Step 3: I develop a simple web site with JS, HTML, and CSS.

This web site includes a fish now button, a schedule for feeding a login part which ai can develop it in future if is necessary and temperature part for showing the temperature and humidity.



Step 4:

I wrote the code for connecting the main board to the Micro Servo and temperature I used C++ language for this code:

```
#include <WiFi.h>
#include <FirebaseESP32.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <Servo.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11 // DHT 11

DHT dht(DHTPIN, DHTTYPE);
Servo servo;

// Replace with THE actual credentials data on firebase dataset
#define FIREBASE_HOST "firebase-project-id.firebaseio.com"
#define FIREBASE_AUTH "firebase-database-secret"
#define WIFI_SSID "SSID" // WIFI NAME
#define WIFI_PASSWORD "PASSWORD"

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800);

FirebaseData timer, feed, tempData;
```

```
String stimer;
String Str[] = {"00:00", "00:00", "00:00"};
int i, feednow = 0;

void setup() {
  Serial.begin(9600);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.print("Connected: ");
  Serial.println(WiFi.localIP());

  timeClient.begin();
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
  Firebase.reconnectWiFi(true);
  servo.attach(23); // Pin attached to GPIO

  dht.begin();
}

void loop() {
  // Read temperature and humidity
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
  } else {
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.print(" %\t");
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" *C");

    // temp
    Firebase.setFloat(tempData, "temperature", temperature);
    Firebase.setFloat(tempData, "humidity", humidity);
  }

  Firebase.getInt(feed, "feednow");
```

```
  feednow = feed.intData();
  Serial.println(feednow);
  if (feednow == 1) { // Direct Feeding
    servo.writeMicroseconds(1000); // rotate clockwise
    delay(700); // allow to rotate for n micro seconds
    servo.writeMicroseconds(1500); // stop rotation
    feednow = 0;
    Firebase.setInt(feed, "feednow", feednow);
    Serial.println("Fed");
  } else { // Scheduling feed
    for (i = 0; i < 3; i++) {
      String path = "timers/timer" + String(i);
      Firebase.getString(timer, path);
      stimer = timer.stringData();
      Str[i] = stimer.substring(9, 14);
    }
    timeClient.update();
    String currentTime = String(timeClient.getHours()) + ":" + String(timeClient.getMinutes());
    if (Str[0] == currentTime || Str[1] == currentTime || Str[2] == currentTime) {
      servo.writeMicroseconds(1000); // rotate clockwise
      delay(700); // allow to rotate for n micro seconds, you can change this to your need
      servo.writeMicroseconds(1500); // stop rotation
      Serial.println("Success");
      delay(60000);
    }
  }
  Str[0] = "00:00";
  Str[1] = "00:00";
  Str[2] = "00:00";

  delay(2000); // Delay between readings
}
```

Also, I tried to write the code for connecting stepper motor to the main board:

```
#include <WiFi.h>
#include <FirebaseESP32.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <AccelStepper.h>
#include "DHT.h"

Pin configuration for DHT11
#define DHTPIN 4 // Pin connected to the data pin of DHT11
```

```
#define DHTTYPE DHT11 // DHT 11
DHT dht(DHTPIN, DHTTYPE);

// Pin configuration for stepper motor
#define IN1 15
#define IN2 2
#define IN3 4
#define IN4 5

// Define the stepper motor and the number of steps per revolution
AccelStepper stepper(AccelStepper::FULL4WIRE, IN1, IN3, IN2, IN4);

// credentials, if you need my firebase i can provide the auth key and url
#define FIREBASE_HOST "my url firebase project"
#define FIREBASE_AUTH " I have this key in my own acount in firebase"
#define WIFI_SSID ""
#define WIFI_PASSWORD ""

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800);

FirebaseData timer, feed, tempData;
String stimer;
String Str[] = {"00:00", "00:00", "00:00"};
int i, feednow = 0;

void setup() {
  Serial.begin(9600);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.print("Connected: ");
  Serial.println(WiFi.localIP());

  timeClient.begin();
  Firebase.begin(FIREBASE_HOST, FIREBASE_AUTH);
  Firebase.reconnectWiFi(true);
  //dht.begin();

  // Set the maximum speed and acceleration for the stepper motor
  stepper.setMaxSpeed(1000);
```

```
  stepper.setAcceleration(500);
}

// void loop() {
//   // Read temperature and humidity
//   float humidity = dht.readHumidity();
//   float temperature = dht.readTemperature();

//   if (isnan(humidity) || isnan(temperature)) {
//     Serial.println("Failed to read from DHT sensor!");
//   } else {
//     Serial.print("Humidity: ");
//     Serial.print(humidity);
//     Serial.print(" %\t");
//     Serial.print("Temperature: ");
//     Serial.print(temperature);
//     Serial.println(" *C");

//     // Update Firebase with temperature data
//     Firebase.setFloat(tempData, "temperature", temperature);
//     Firebase.setFloat(tempData, "humidity", humidity);
//   }

  Firebase.getInt(feed, "feednow");
  feednow = feed.intData();
  Serial.println(feednow);
  if (feednow == 1) { // Direct Feeding
    stepper.moveTo(200);  // Move 200 steps forward
    stepper.runToPosition();  // Block until the move is complete
    delay(700); // allow to hold for a while
    stepper.moveTo(0);  // Move back to the original position
    stepper.runToPosition();  // Block until the move is complete
    feednow = 0;
    Firebase.setInt(feed, "feednow", feednow);
    Serial.println("Fed");
  } else { // Scheduling feed
    for (i = 0; i < 3; i++) {
      String path = "timers/timer" + String(i);
      Firebase.getString(timer, path);
      stimer = timer.stringData();
      Str[i] = stimer.substring(9, 14);
    }
    timeClient.update();
    String currentTime = String(timeClient.getHours()) + ":" +
String(timeClient.getMinutes());
```

```
    if (Str[0] == currentTime || Str[1] == currentTime || Str[2] == currentTime)
{
      stepper.moveTo(200);  // Move 200 steps forward
      stepper.runToPosition();  // Block until the move is complete
      delay(700); // allow to hold for a while
      stepper.moveTo(0);  // Move back to the original position
      stepper.runToPosition();  // Block until the move is complete
      Serial.println("Success");
      delay(60000);
    }
  }
  Str[0] = "00:00";
  Str[1] = "00:00";
  Str[2] = "00:00";

  delay(2000); // Delay between readings
}
```

Step 5:

I put an authentication method on my firebase project including email and password for more security and safety. The email and password is my university account.

Step 6:

To connecting the pin from my main board to the Micro servo I used these pins in my main board:

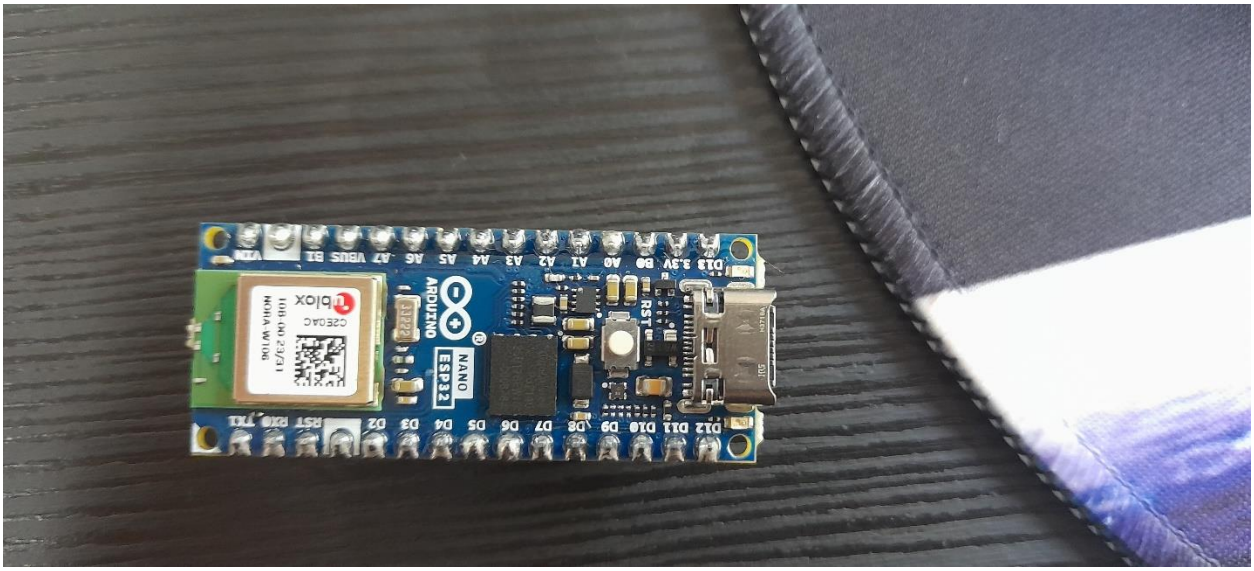
1. Signal Pin: Connect to B1 (confirm the actual GPIO number, e.g., GPIO 23).
2. Power Pin: Connect to VIN.
3. Ground Pin: Connect to GND

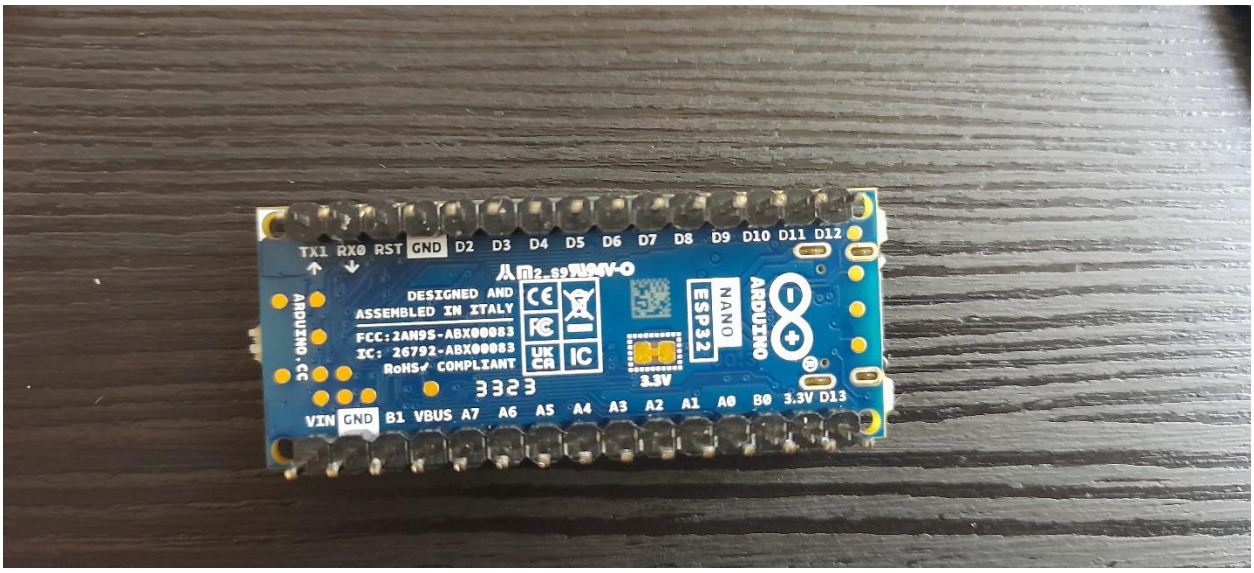
To connecting the temperature to the main board I used these pins:

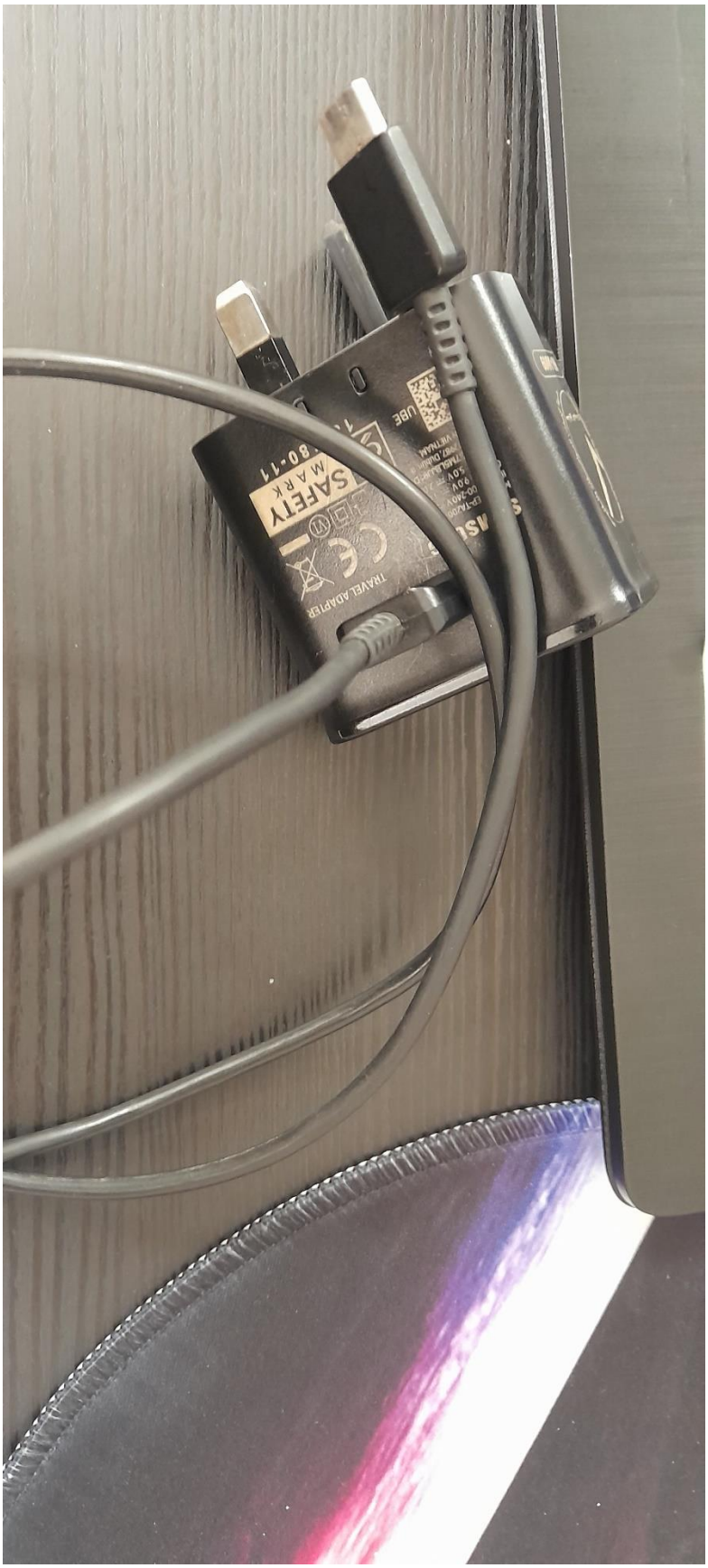
1. VCC: Connect to the 3.3V pin on the ESP32.
2. GND: Connect to the GND pin on the ESP32.
3. Data: Connect to a GPIO pin on the ESP32 (e.g., GPIO 4).
4. But unfortunately, my temperature did not work I guess it damaged and burned and this was because of the voltage.

Final step:

I used the 3D printer at the university laboratory with help and support of the laboratory technician and made some 3d part with PLA texture. I put some picture from different parts of my project.



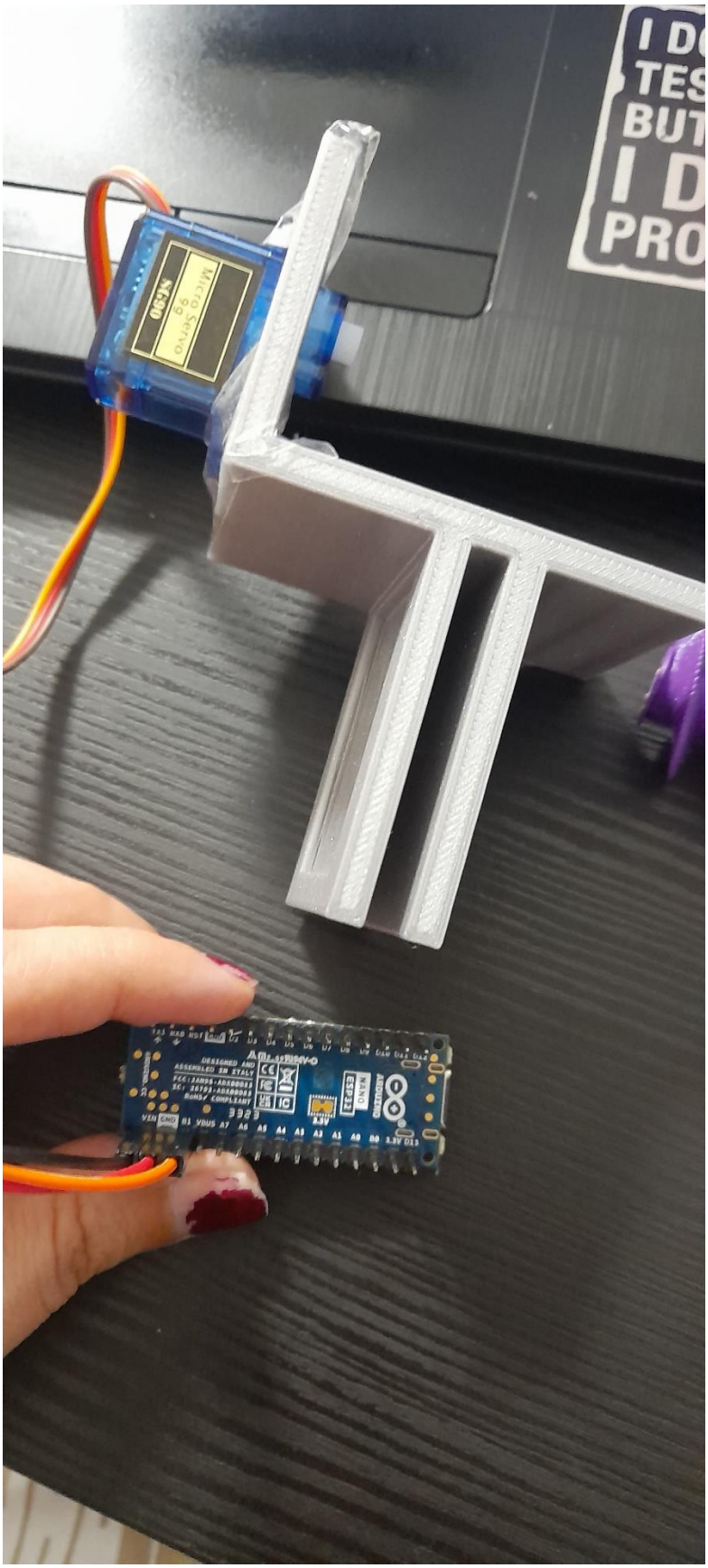


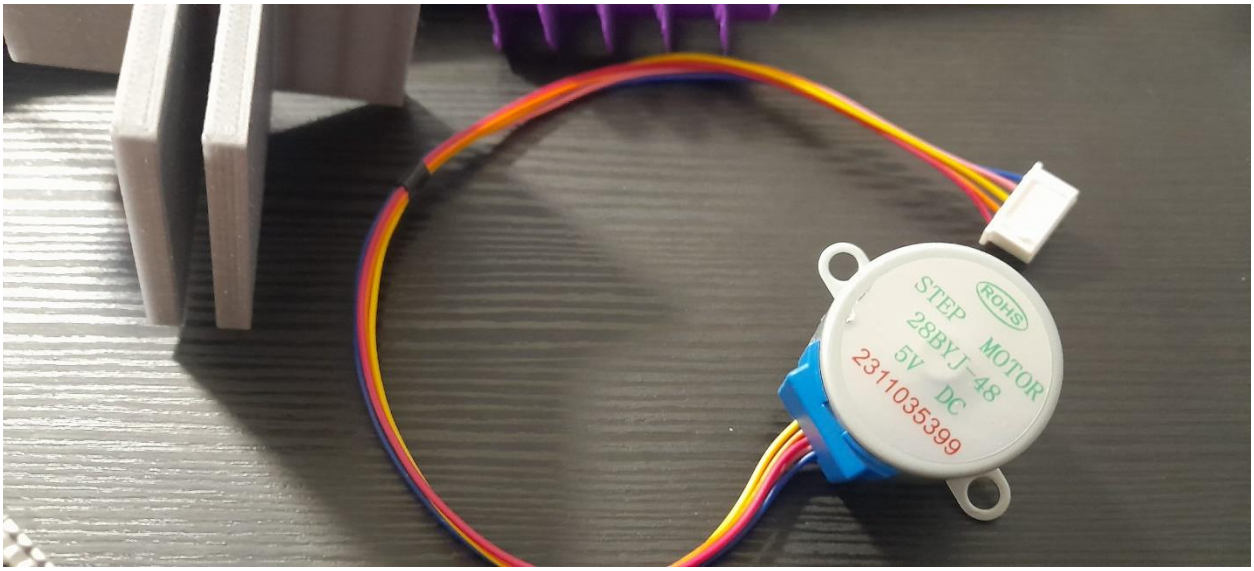

SAFETY
MARK
TRAVEL ADAPTER

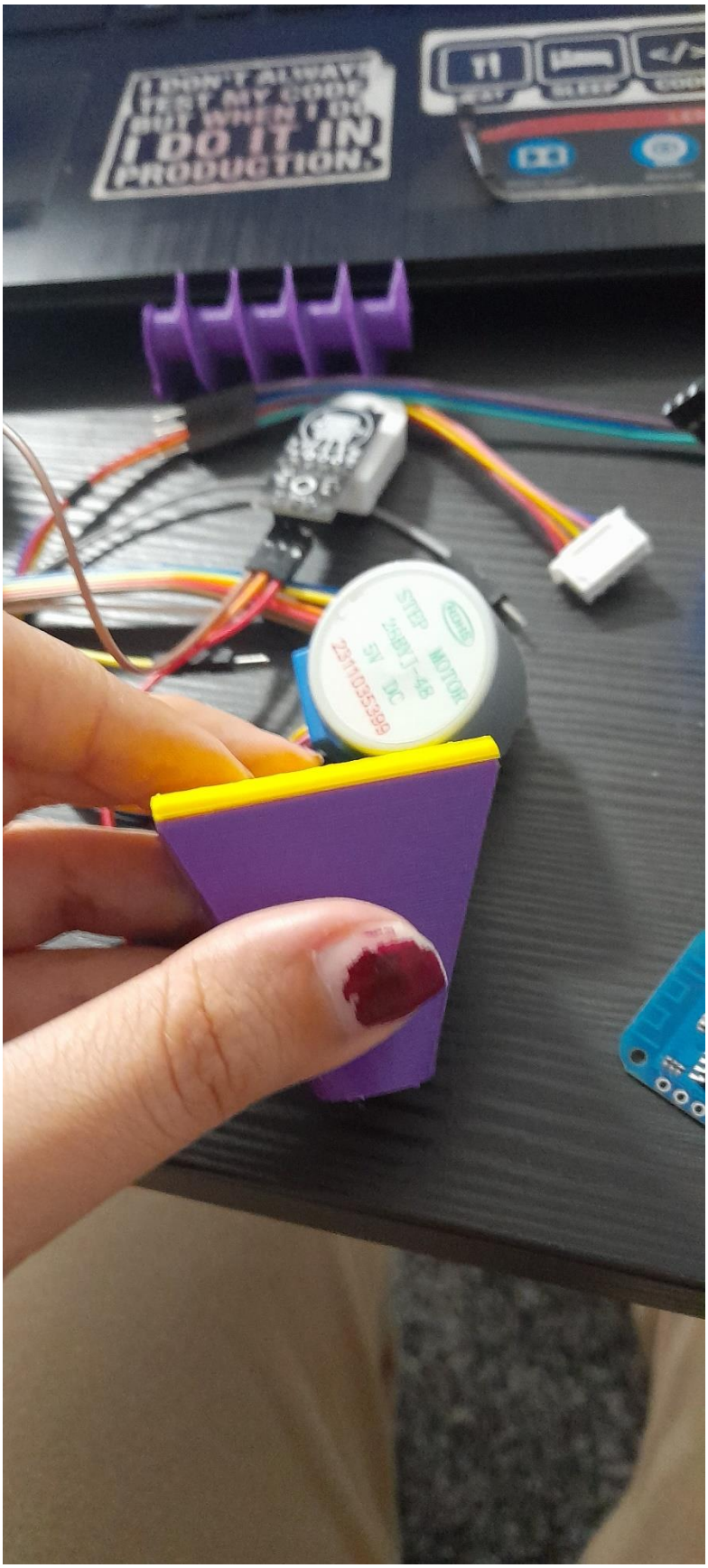

SAMSUNG
TRAVEL ADAPTER
MODEL : EP-TA200
INPUT : 100-240 V ~ 50-60 Hz 0.5 A
OUTPUT : 9.0 V ⎓ 1.67 A or
5.0 V ⎓ 2.0 A
S/N : R37M5LBJ91DK3
SAFETY
MARK
181180-11

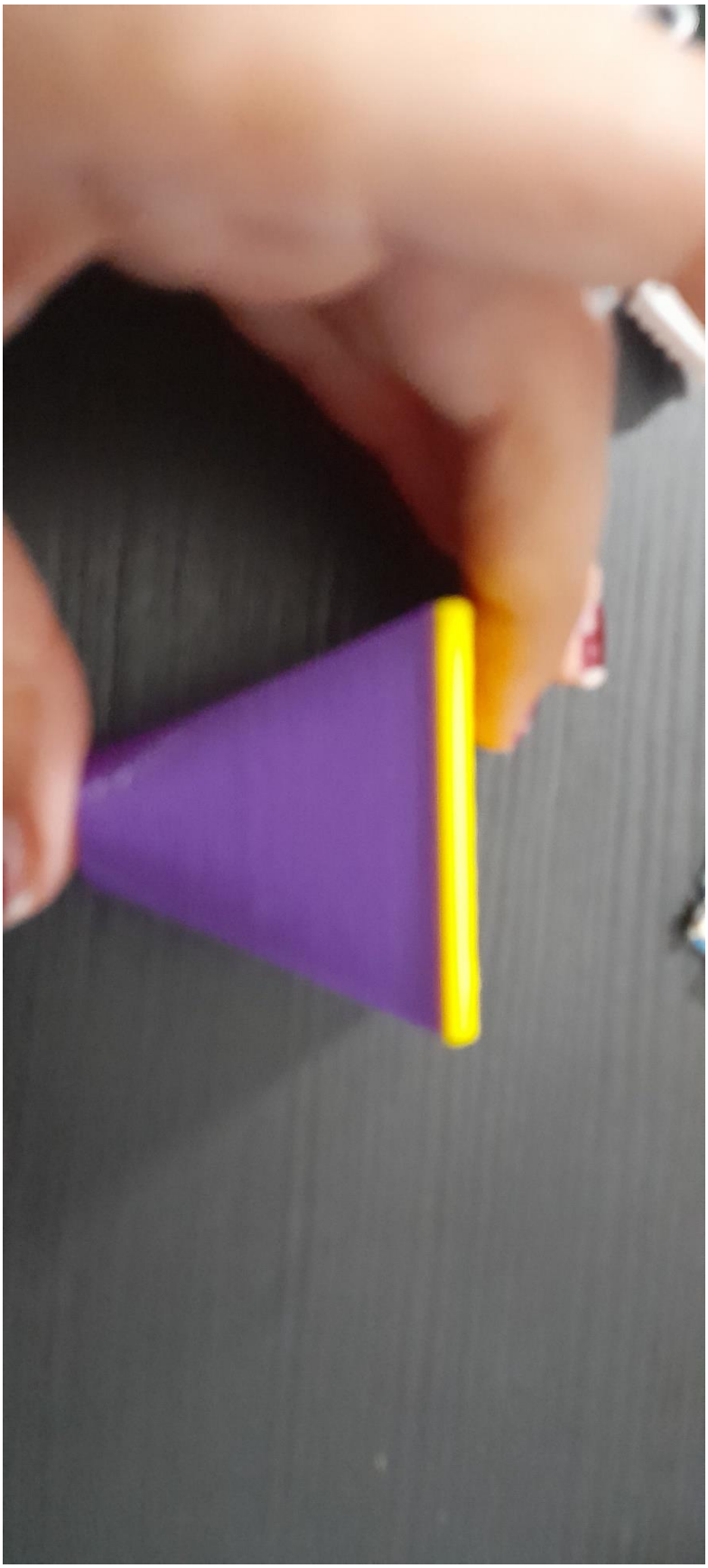

I DON'T ALW
TEST MY C
BUT WHEN
I DO IT
PRODUCTI


Micro Servo
9g

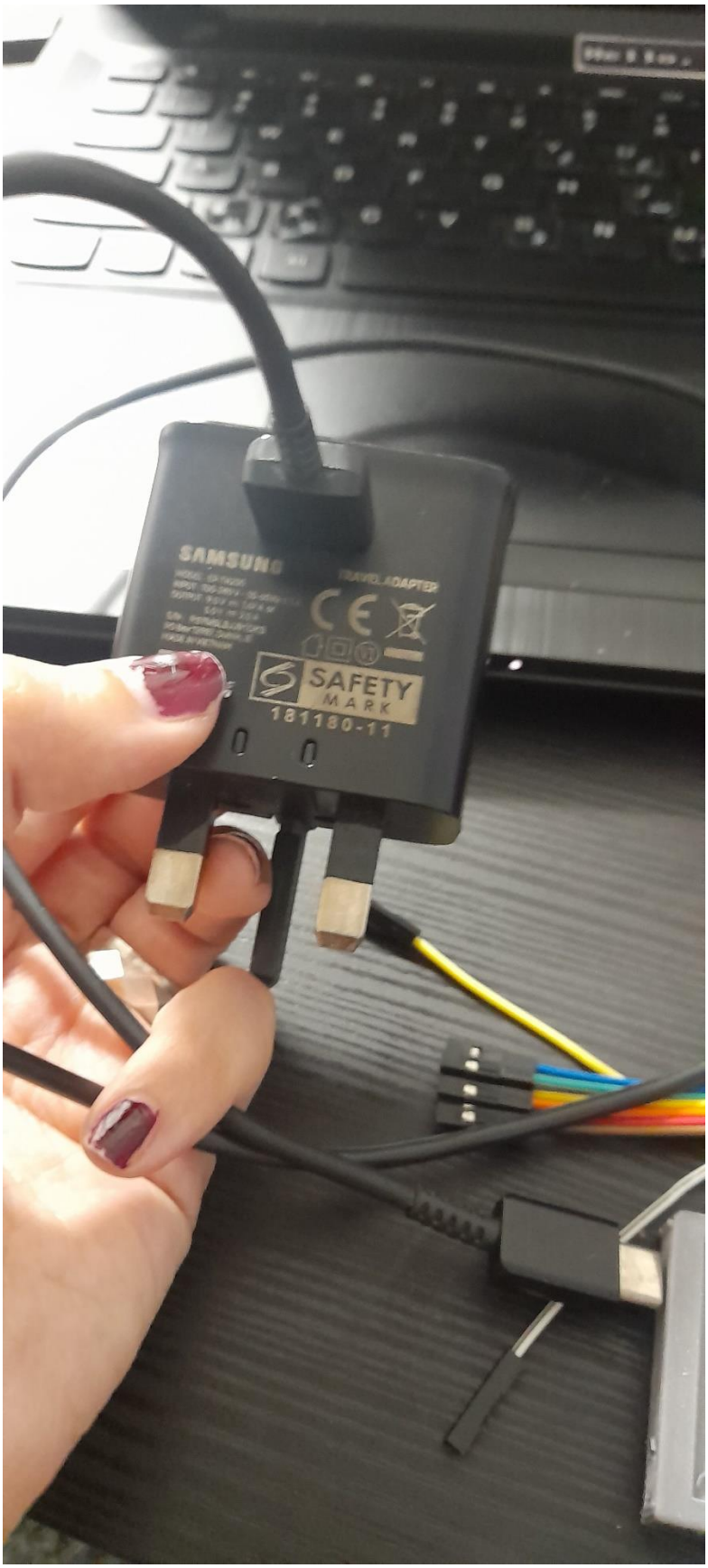

Micro Servo 9g


ROHS
STEP MOTOR
28BYJ-48
5V DC
2311035399


I DON'T ALWAYS
TEST MY CODE
BUT WHEN I DO
I DO IT IN
PRODUCTION.




docs.arduino.cc/nano-esp32
IoT MADE EASY
Connectivity
Wi-Fi®
Bluetooth® LE
Chip
ESP32-S3
Flash Memory
8MB INTERNAL
16MB EXTERNAL
RAM
512KB INTERNAL
Voltage
5V USB
3.3V OPERATING
6-21V INPUT
Pinout
14 DIGITAL
8 ANALOG
Dimensions
43.1 x 17.78mm
COMPATIBLE WITH ARDUINO CLOUD
FCC ID: 2AN95-ABX00083
IC: 26792-ABX00083


SAMSUNG
TRAVEL ADAPTER
SAFETY
MARK
181180-11

At the end, I really enjoy to working on this practical project although there is some errors and difficulties but thank you Dr. AlZu'bi for help and valuable information.

References:

[1] https://www.arduino.cc/en/software

[2] https://firebase.google.com/

[3] https://www.microsoft.com/en/microsoft-365/word?market=qa

[4] https://www.amazon.co.uk/Eheim-Everyday-Programmable-Automatic-Dispenser

[5] https://www.amazon.co.uk/AmazonBasics-Electronic-Timed-Feede

[6] https://www.amazon.co.uk/Fish-Mate-F14-Aquarium-Feede